DFSha: Sistema de Archivos Distribuido de Alta Disponibilidad

Entregable 1 — Definición del servicio y arquitectura propuesta

ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos — 2026-2
Universidad EAFIT

Integrantes: Sara Isabel Vasquez – Estefania Ramirez
Fecha: 13/09/2026

---

1. Objetivo general

Diseñar e implementar DFSha, un sistema de archivos distribuido basado en bloques, con arquitectura Cliente/Servidor compuesta por un nodo central de metadatos y múltiples nodos de almacenamiento, que permita a los clientes almacenar y recuperar archivos grandes (del orden de GB a TB) de forma distribuida entre varios nodos, garantizando alta disponibilidad, rendimiento, transparencia y seguridad tanto en la escritura como en la lectura.

2. Definición del servicio

El servicio corresponde a la **Opción 1: arquitectura Cliente/Servidor (C/S)**, en la cual el lado servidor se implementa mediante composición y distribución del servicio (S2S) dentro de un sistema autónomo administrado por una misma organización. DFSha sigue un enfoque de **DFS por bloques**, inspirado en la arquitectura de HDFS, y está compuesto por tres tipos de actores:

- **Cliente DFSha:** aplicación (CLI y/o API) que consume el servicio. No forma parte del backend distribuido; interactúa con él bajo el modelo cliente/servidor.
- **ControlNode** (equivalente al NameNode de HDFS): servidor central que mantiene los metadatos del sistema — estructura de directorios, qué archivos existen, en qué bloques se dividió cada uno y en qué DataNodes se ubica cada bloque y sus réplicas.
- **DataNode:** servidores que almacenan físicamente los bloques de datos y atienden operaciones de lectura/escritura directa sobre esos bloques.

2.1 Requerimientos funcionales (RF)

- **RF1 — Gestión del sistema de archivos:** mkdir, rmdir, ls, rm, cd sobre una jerarquía de directorios tipo Linux, mantenida por el ControlNode.
- **RF2 — Transferencia de archivos:** put <archivo> (particiona el archivo en bloques y los distribuye en los DataNodes) y get <archivo> (recupera y reconstruye el archivo a partir de los bloques).
- **RF3 — Acceso a archivos:** primitivas de bajo nivel open(), read(), write(), close(), lock() sobre bloques individuales, para soportar acceso concurrente y no solo transferencia completa del archivo.
- **RF4 — Autenticación básica:** cada cliente accede mediante usuario/contraseña; cada cliente gestiona solo sus propios archivos (aislamiento por usuario).
- **RF5 — API de nodos:** tanto el ControlNode como los DataNodes exponen una API (REST/gRPC) para que el cliente y los demás nodos interactúen con ellos.

2.2 Requerimientos no funcionales (RNF)

- **RNF1 — Escalabilidad:** soportar crecimiento en número de usuarios, archivos y DataNodes sin cambios estructurales; permitir agregar/quitar DataNodes en caliente.

- **RNF2 — Alta disponibilidad:** cada bloque se replica en al menos N DataNodes (factor de replicación configurable, ej. N=3), de forma que la caída de un nodo no implique pérdida de datos ni del servicio.
- **RNF3 — Consistencia de datos:** dado que existen réplicas, se define una política de consistencia tipo quorum (ej. escritura confirmada solo cuando W réplicas responden).
- **RNF4 — Particionamiento:** cada archivo se divide en bloques de tamaño fijo (ej. 64 MB) distribuidos entre DataNodes mediante un algoritmo que balancee la carga, evitando que un mismo nodo concentre siempre los bloques.
- **RNF5 — Rendimiento:** acceso concurrente a distintos bloques de un mismo archivo desde distintos DataNodes en paralelo, evitando que el ControlNode sea cuello de botella.
- **RNF6 — Seguridad:** cifrado en tránsito (TLS) entre cliente-servidor y entre nodos; cifrado en reposo de los bloques almacenados; control de acceso por usuario, con posibilidad de evolucionar a API Keys / 2FA.
- **RNF7 — Transparencia:** el cliente accede mediante una CLI/API propia, sin necesidad de conocer en qué DataNode físico está cada bloque.

3. Arquitectura del sistema

Se propone una arquitectura Cliente/Servidor con composición interna Maestro–Trabajadores (Master–Workers), inspirada en HDFS. El ControlNode actúa como maestro: es el único punto de entrada para metadatos y decide en qué DataNodes va cada bloque. Los DataNodes actúan como trabajadores: solo almacenan bloques y sirven datos, sin tomar decisiones de particionamiento. El Cliente no transfiere los datos a través del ControlNode; este le indica dónde están los bloques y el Cliente transfiere los datos directamente con los DataNodes correspondientes, evitando que el ControlNode se convierta en cuello de botella (RNF5).

Para la alta disponibilidad del propio ControlNode (RNF2) se propone un esquema activo/standby: un ControlNode primario activo y uno o más ControlNode en modo standby, sincronizados periódicamente, que asumen el rol de maestro ante una falla del primario.

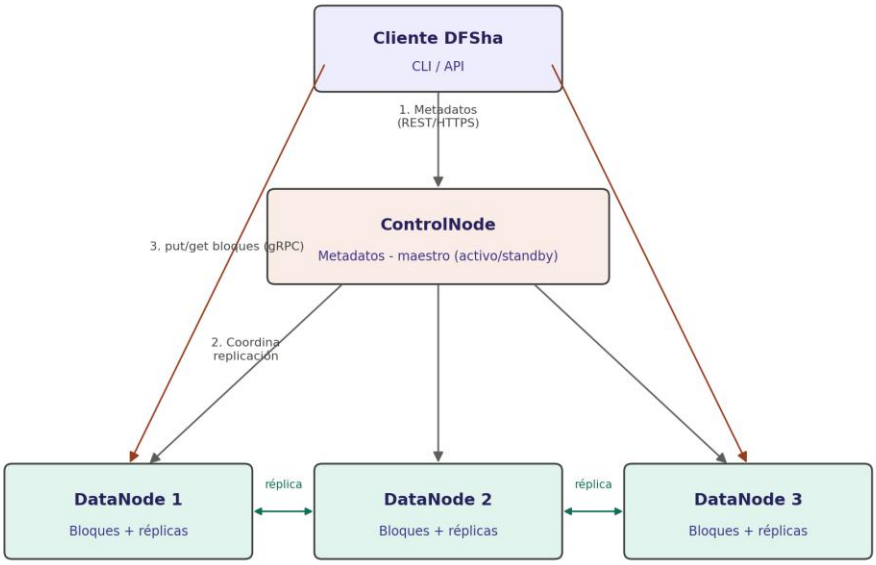


Figura 1. Arquitectura propuesta para DFSha: Cliente, ControlNode y DataNodes.

3.1 Componentes

Componente	Rol	Responsabilidad
Cliente DFSha	CLI / API	Autenticación, comandos de FS (ls, mkdir, etc.), put/get; habla con ControlNode para metadatos y con DataNodes para datos
ControlNode	Maestro	Metadatos: estructura de directorios, mapeo archivo→bloques→DataNodes, gestión de usuarios, coordinación de réplicas y rebalanceo
DataNode	Trabajador	Almacena bloques físicos, atiende lectura/escritura de bloques, reporta estado/heartbeat al ControlNode

3.2 Particionamiento y replicación

- Cada archivo se divide en chunks/bloques de tamaño fijo (ej. 64 MB).
- Cada bloque se replica con un factor de replicación N (ej. N=3) en DataNodes distintos.
- Algoritmo de asignación: round-robin ponderado por carga — el ControlNode mantiene el estado de uso (espacio, número de bloques) de cada DataNode y asigna los nuevos bloques a los nodos con menor carga relativa (se descarta la asignación puramente aleatoria por concentrar carga en los mismos nodos).
- Rebalanceo: al agregar nuevos DataNodes, el ControlNode detecta el desbalance vía heartbeats y dispara migración de bloques hacia los nodos nuevos.
- Ante la caída o retiro de un nodo, el ControlNode detecta réplicas por debajo del factor N y ordena re-replicación en otros DataNodes disponibles.

4. Comunicaciones de red y protocolos

El servicio se ejecuta sobre Internet, con nodos comunicándose mediante REST y/o gRPC según el tipo de canal. La siguiente tabla resume la propuesta inicial de protocolos:

Canal	Protocolo propuesto	Justificación
Cliente ↔ ControlNode	REST sobre HTTPS	Operaciones de metadatos (ls, mkdir, autenticación, localización de archivos) son solicitud/respuesta simples
Cliente ↔ DataNode	gRPC	Transferencia de bloques grandes; gRPC es más eficiente para streaming de datos binarios
ControlNode ↔ ControlNode	REST/gRPC interno	Sincronización de metadatos entre el nodo primario y el standby (alta disponibilidad)
ControlNode ↔ DataNode	REST/gRPC + heartbeats	Envío de comandos de replicación y recepción
DataNode ↔ DataNode	gRPC	Copia de bloques entre nodos para cumplir el factor de replicación, sin pasar por el ControlNode ni el cliente

Seguridad de las comunicaciones (RNF6): todos los canales anteriores deben operar sobre TLS, y cada mensaje —en particular Cliente↔ControlNode— debe incluir el token o credencial del usuario autenticado.

5. Próximos pasos

- Hito 2: diseño detallado de la arquitectura distribuida y especificación formal de las APIs (endpoints, contratos de mensajes).
- Hito 3: implementación de alta disponibilidad, replicación, consistencia y seguridad.
- Hito 4: entrega final, pruebas de carga/concurrencia y documentación completa.